

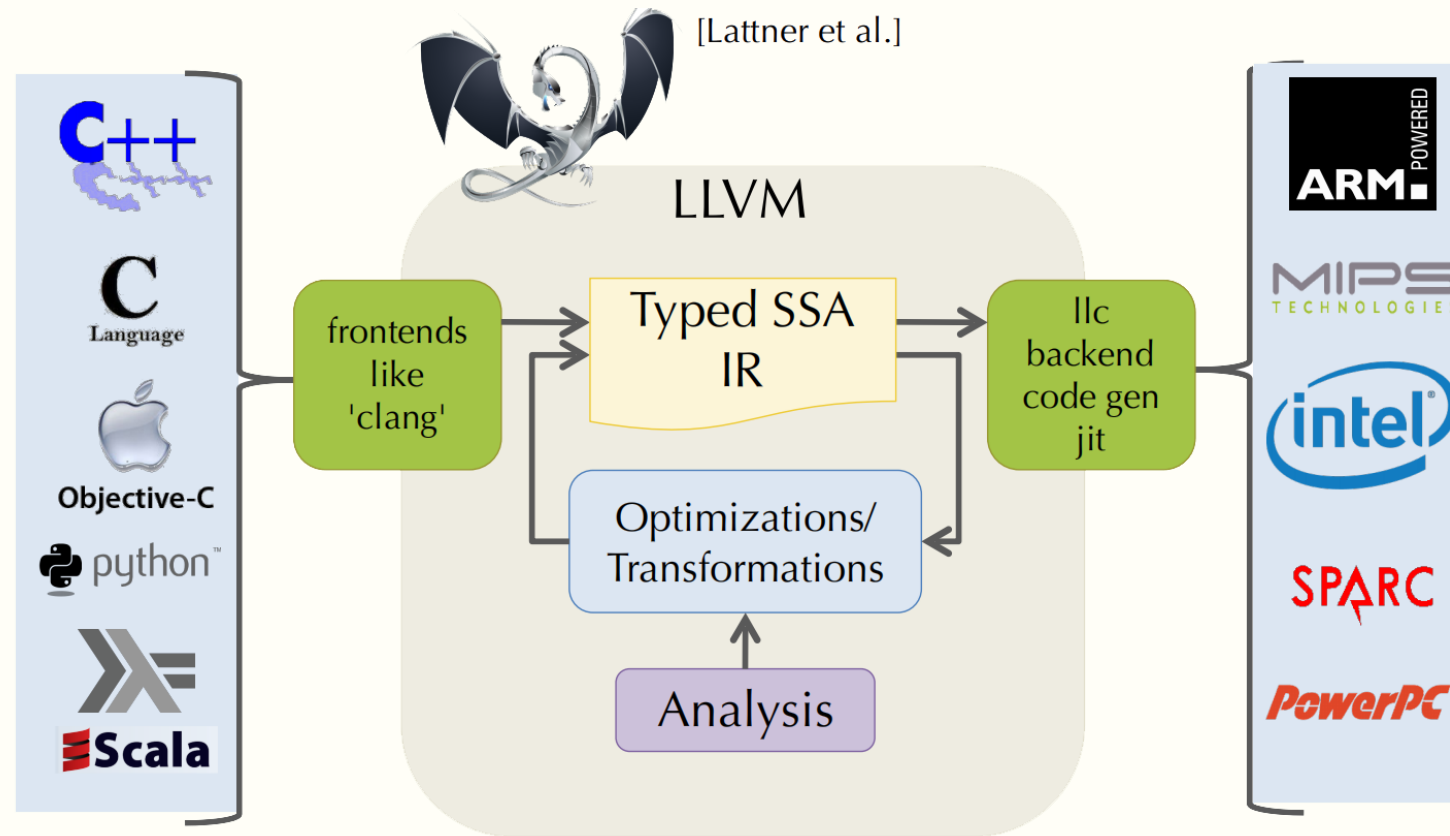


《编译原理》

第3讲：中间表示

何冬杰
重庆大学

IR: The Backbone of Modern Compilation



LLVM Compiler Infrastructure

Low-Level Virtual Machine (LLVM)



- ❑ Open-Source Compiler Infrastructure

- See llvm.org for full documentation

- ❑ Created by **Chris Lattner** (advised by Vikram Adve) at UIUC

- LLVM: An infrastructure for Multi-stage Optimization, 2002
 - LLVM: A Compilation Framework for Lifelong Program Analysis and Transformation, 2004

- ❑ 2005: Adopted by Apple for XCode 3.1

- ❑ Front ends:

- llvm-gcc (drop-in replacement for gcc)
 - Clang: C, objective C, C++ compiler supported by Apple
 - various languages: Swift, ADA, Scala, Haskell, ...

- ❑ Back ends:

- x86 / Arm / PowerPC / etc.

- ❑ Used in many academic/research projects



Why intermediate representations (IRs)?

❑ Syntax-directed translations work fine for simple languages:

- Input syntax uniquely determines output
- No complex analysis or code transformation is done

❑ But...

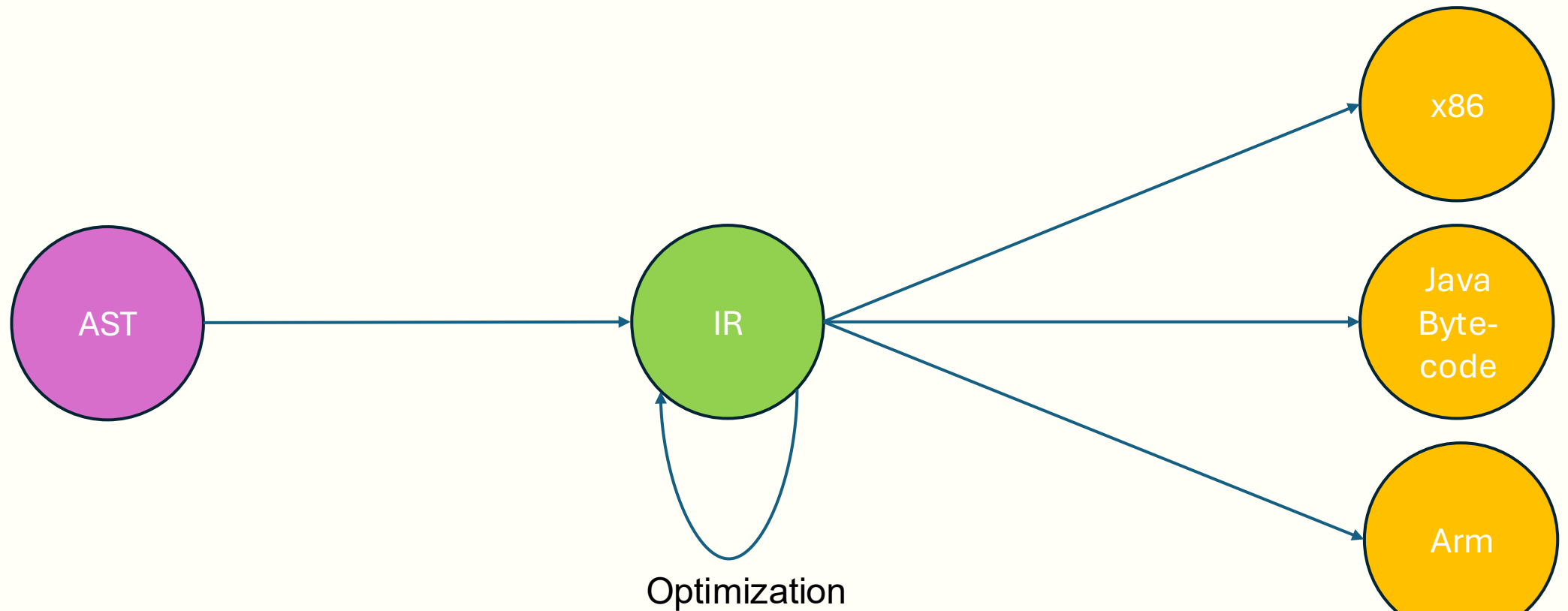
- Resulting code quality is poor
- Richer source language features are hard to encode
 - Structured data types, objects, first-class functions, etc.
- Hard to optimize the resulting assembly code
 - Representation is too concrete – e.g. it has committed to using certain registers and the stack
 - Only a fixed number of registers
 - Some instructions have restrictions on where the operands are located
- Control-flow is not structured
 - Arbitrary jumps from one code block to another
 - Implicit fall-through makes sequences of code non-modular (i.e. you can't rearrange sequences of code easily)

❑ Retargeting the compiler to a new architecture is hard

- Target assembly code is hard-wired into translation

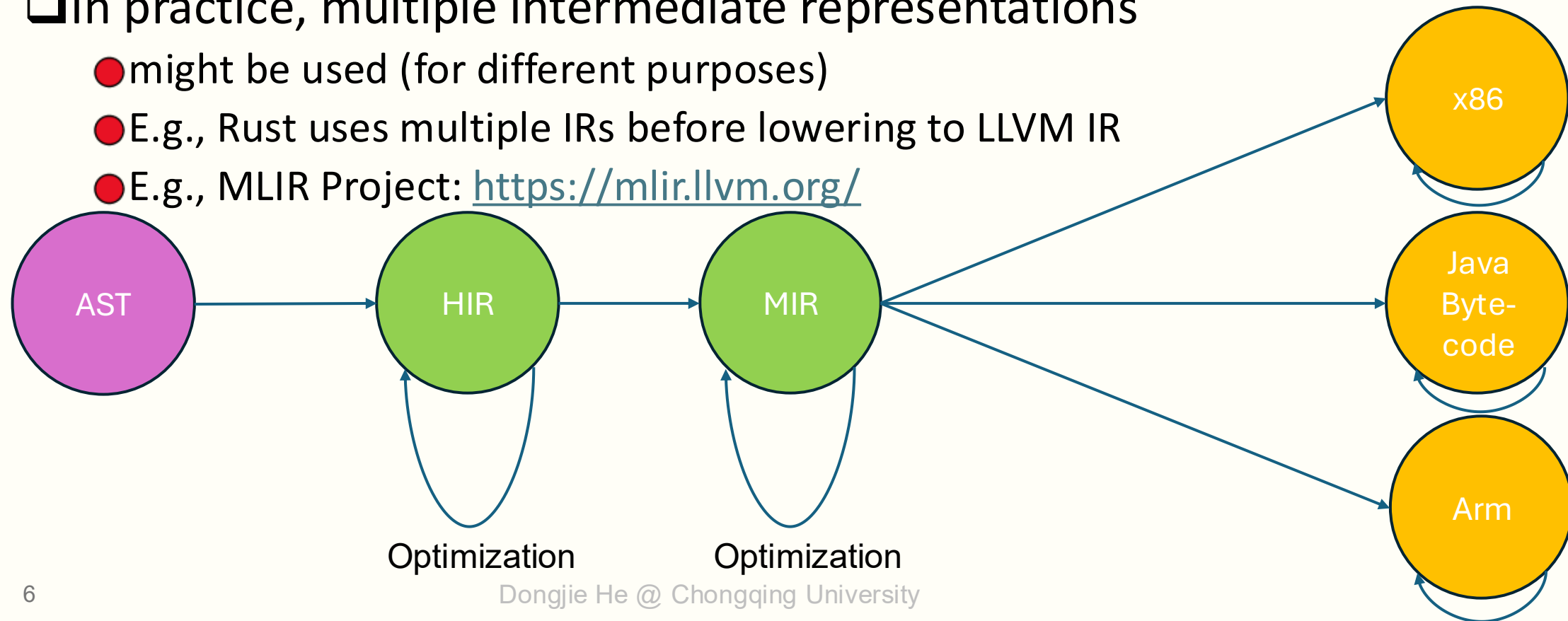
Intermediate Representations (IRs)

- ❑ Abstract machine code: hides details of the target architecture
- ❑ Allows machine independent code generation and optimization.



Multiple IRs

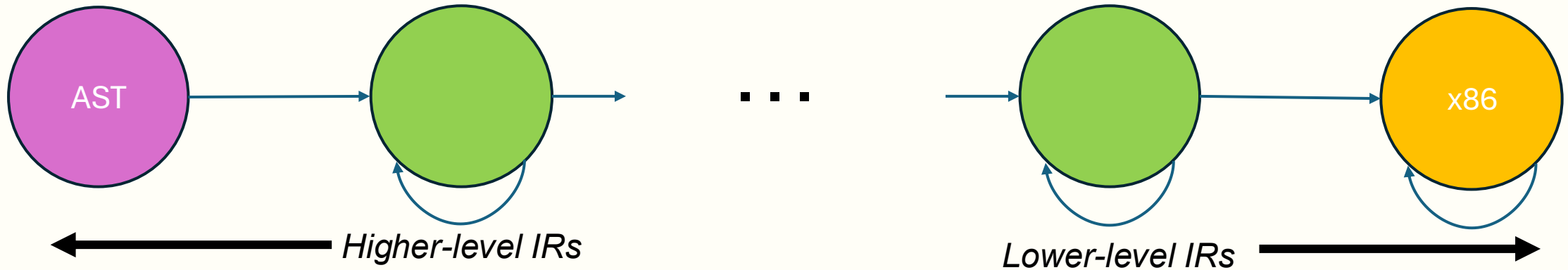
- ❑ Goal: get program closer to machine code without losing the information needed to do analysis and optimizations
- ❑ In practice, multiple intermediate representations
 - might be used (for different purposes)
 - E.g., Rust uses multiple IRs before lowering to LLVM IR
 - E.g., MLIR Project: <https://mlir.llvm.org/>



What makes a good IR?

- ❑ Easy translation target (from the level above)
 - ❑ Easy to translate (to the level below)
 - ❑ Narrow interface
 - Fewer constructs mean simpler phases/optimizations
 - ❑ Example:
 - Source language might have **while**, **for**, **foreach**, **do-while**, **do-until** loops, ...
 - IR might have only **while** loops and sequencing
 - Translation eliminates **for** and **foreach**
- $$\llbracket \text{for}(\text{pre}; \text{cond}; \text{post}) \{ \text{body} \} \rrbracket = \llbracket \text{pre}; \text{while}(\text{cond}) \{ \text{body}; \text{post} \} \rrbracket$$
- Here $\llbracket \text{cmd} \rrbracket$ denotes the translation or compilation of the command cmd.

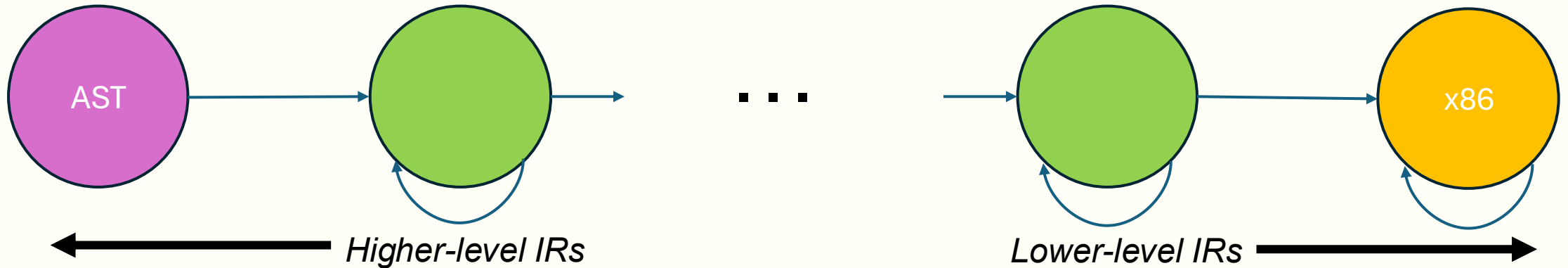
IRs at the extreme



□ High-level IRs

- Abstract syntax + new node types not generated by the parser
 - e.g. Type checking information or disambiguated syntax nodes
- Typically preserves the high-level language constructs
 - Structured control flow, variable names, methods, functions, etc.
 - May do some simplification (e.g. convert for to while)
- Allows high-level optimizations based on program structure
 - e.g. inlining “small” functions, reuse of constants, etc.
- Useful for semantic analyses like type checking

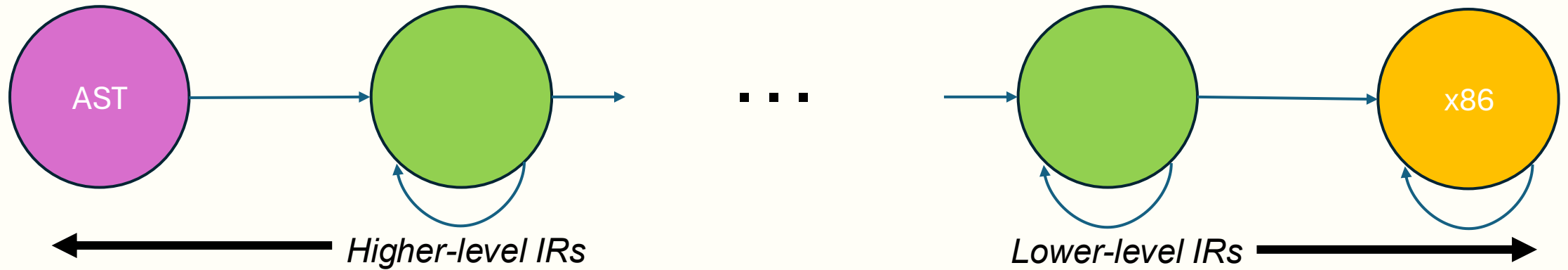
IRs at the extreme



□ Low-level IRs

- Machine dependent assembly code + extra pseudo-instructions
 - e.g. a pseudo instruction for interfacing with garbage collector or memory allocator (parts of the language runtime system)
 - e.g. (on x86) an **imulq** instruction that doesn't restrict register usage
- Source structure of the program is lost:
 - Translation to assembly code is straightforward
- Allows low-level optimizations based on target architecture
 - e.g. register allocation, instruction selection, memory layout, etc.

IRs at the extreme



❑ What's in between?

● Mid-level Intermediate representations (IRs)

Mid-level IRs

- ❑ Intermediate between AST (abstract syntax) and assembly
- ❑ May have unstructured jumps, abstract registers or memory locations
- ❑ Convenient for translation to high-quality machine code
 - Example: all intermediate values might be named to facilitate optimizations that attempt to minimize stack/register usage
- ❑ Many Varieties:
 - **Quadruples**: $a = b \text{ OP } c$ (“three address form”)
 - **SSA**: variant of quadruples where each variable is assigned exactly once
 - Easy dataflow analysis for optimization, e.g. LLVM: industrial-strength IR, based on SSA
 - **Triples**: $\text{OP } a \text{ } b$
 - “Name” of result is implicit
 - Useful for instruction selection on X86 via “tiling”
 - **Stack-based**: Easy to generate, e.g., Java Bytecode, UCODE

Mid-level IRs: Example

IR	$a = b + a * c$	Key Feature
Quadruples (Three Address Code)	$t1 = a * c; a = b + t1$	The result has an explicit name (t1)
SSA	$t1 = a * c; a1 = b + t1$	Each variable is assigned exactly once.
Triples	$(0) * c\ d; (1) + b\ (0)$	Results are referenced by instruction position.
Stack-based	Load b; load a; load c; mul; add; store a	Operands are implicit on the stack.

LLVMLite: a simplified LLVM IR

- ❑ We develop a lightweight subset of the LLVM IR
 - Rich enough to support many modern language features
 - Easy to lower to assembly
 - Bridges the gap to real-world compilers such as LLVM
- ❑ LLVMLite IR
 - No modules or SSA
 - Typed operands and instructions
 - Basic blocks and control flow
 - load/store operations
- ❑ Future work
 - AST → LLVMLite IR translation
 - LLVMLite IR → Assembly translation

An IR Program

- ❑ An IR program consists of global variables followed by functions.

```
type gid = string
type program = {
  gvdefs : (gid * gvdef) list;
  fndefs : (gid * fndef) list
}
```

- ❑ Global variable definition:

- Data type, initializer, and a flag indicating whether the value is mutable.

```
type gvdef = {
  ty : ty; init : ginit; const : bool
}
```

- ❑ Function definition:

- The body is presented as a control-flow graph (CFG).

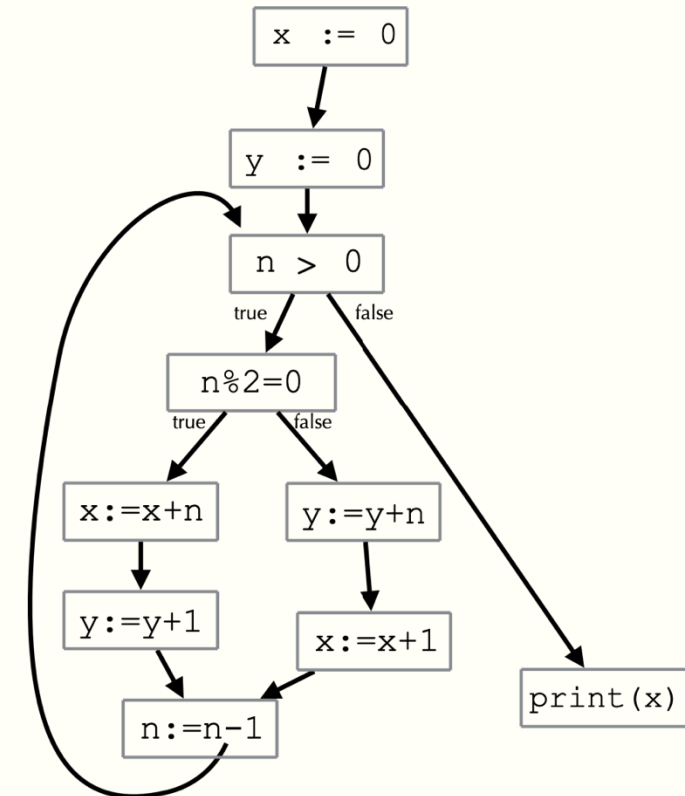
```
type fndef = {
  param : (ty * uid) list;
  ret_ty : ty;
  cfg : cfg
}
```

Control-Flow Graphs

□ Graphical representation of a program

- Edges in graph represent control flow: how execution traverses a program
- Nodes represent statements

```
x := 0;  
y := 0;  
while (n > 0) {  
  if (n % 2 = 0) {  
    x := x + n;  
    y := y + 1;  
  }  
  else {  
    y := y + n;  
    x := x + 1;  
  }  
  n := n - 1;  
}  
print(x);
```



Basic Blocks

□ Sequences of statements such that:

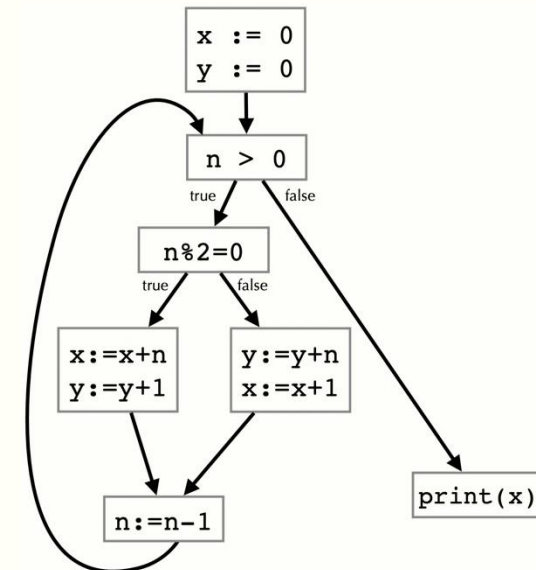
- Can be entered only at the beginning of the block
- Can be exited only at the end of the block
 - Exit by branching, by unconditional jump to another block, or by returning from a function

□ We will require that nodes of a control flow graph are basic blocks

- Basic blocks simplify representation and analysis

```
x := 0;
y := 0;
while (n > 0) {
  if (n % 2 = 0) {
    x := x + n;
    y := y + 1;
  }
  else {
    y := y + n;
    x := x + 1;
  }
  n := n - 1;
}
print(x);
```

CFG



LLVM Basic Blocks and Control-Flow Graphs

❑ LLVM enforces (some of) the basic block invariants syntactically.

❑ Representation in OCaml:

```
type uid = string
type block = {
  insns : (uid * insn) list;
  term : terminator
}
```

```
type lbl = string
type terminator =
  | Ret of (ty * opn) option
  | Br of lbl
  | Cbr of opn * lbl * lbl
```

❑ A CFG is represented as a list of labelled basic blocks with these invariants:

- No two blocks have the same label
- All terminators mention only labels that are defined among the set of basic blocks
- There is a distinguished, unlabeled, entry block:

```
type cfg = block * (lbl * block) list
```

Example LLVM Code

□ LLVM offers a textual representation of its IR (*.ll)

- Decorates values with type information

- E.g., i64, i64*, i1

- Permits numeric identifiers

- Has alignment annotations

- Keeps track of entry edges:

- preds = %5, %0

```
#include <stdio.h>
#include <stdint.h>

int64_t factorial(int64_t n) {
    int64_t acc = 1;
    while (n > 0) {
        acc = acc * n;
        n = n - 1;
    }
    return acc;
}
```

factorial64.c

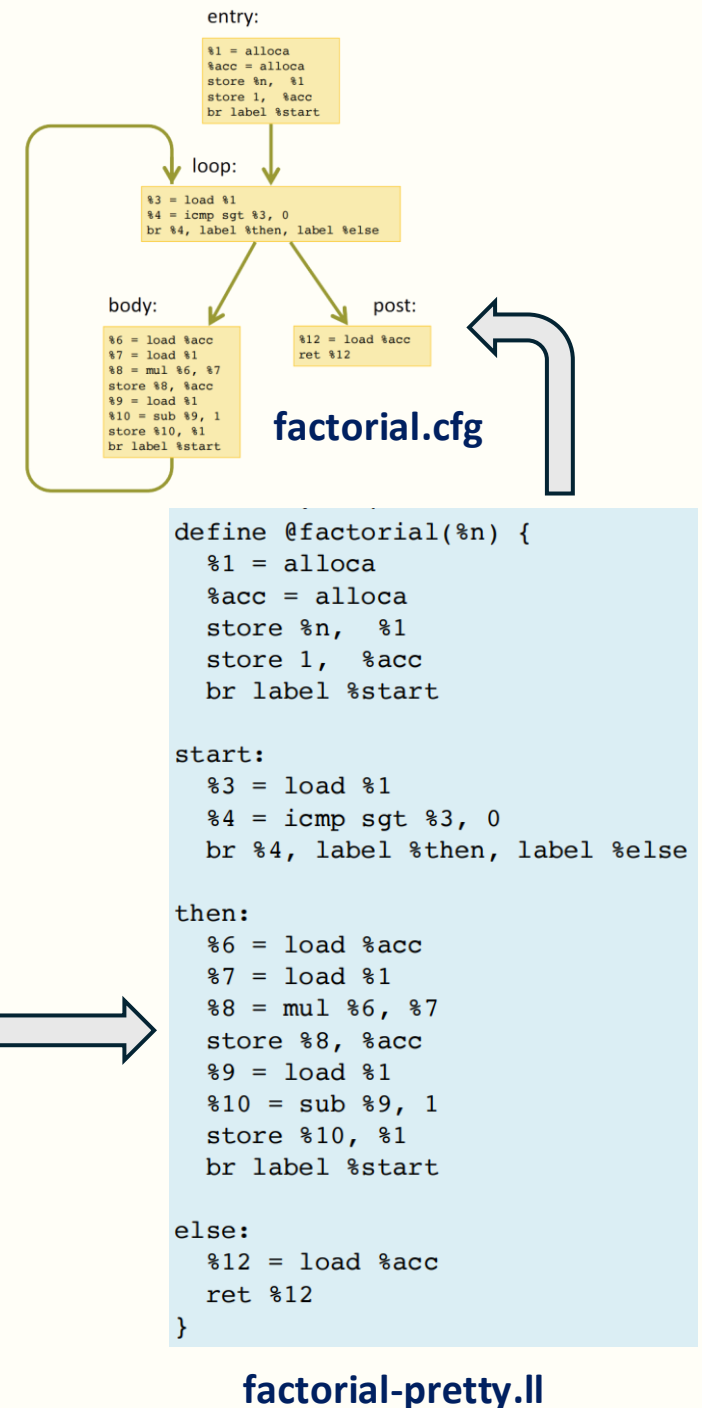
```
; Function Attrs: nounwind ssp
define i64 @factorial(i64 %n) #0 {
    %1 = alloca i64, align 8
    %acc = alloca i64, align 8
    store i64 %n, i64* %1, align 8
    store i64 1, i64* %acc, align 8
    br label %2

; <label>:2                                ; preds = %5, %0
    %3 = load i64* %1, align 8
    %4 = icmp sgt i64 %3, 0
    br i1 %4, label %5, label %11

; <label>:5                                ; preds = %2
    %6 = load i64* %acc, align 8
    %7 = load i64* %1, align 8
    %8 = mul nsw i64 %6, %7
    store i64 %8, i64* %acc, align 8
    %9 = load i64* %1, align 8
    %10 = sub nsw i64 %9, 1
    store i64 %10, i64* %1, align 8
    br label %2

; <label>:11                               ; preds = %2
    %12 = load i64* %acc, align 8
    ret i64 %12
}
```

factorial.ll



Operators, Operands, and Instructions

❑ Binary operators: +, -, *, /, %, &&, ||, ^, <<, >>

```
type bop = Add | Sub | Mul | Div | Rem | And | Or | Xor | Shl | AShr
```

❑ Comparison operators: =, <, >, <=, >=, !

```
type cmpop = Eq | Lt | Gt | Le | Ge | Ne
```

❑ Type casting operators: Zero/Sign Extension, Truncation

```
type castop = ZExt | SExt | Trunc
```

❑ Instructions:

❑ Operands:

```
type opn = Id of uid | Const of int64 | Global of gid |  
Null
```

```
type fn_ty = { params : ty list; ret : ty }  
type insn = Binop of bop * ty * opn * opn  
          | ICmp of cmpop * ty * opn * opn  
          | Alloca of ty * opn  
          | Load of ty * opn  
          | Store of ty * opn * opn  
          | Gep of ty * opn * typed_opn list  
          | Cast of castop * typed_opn * ty  
          | Call of fn_ty * opn * opn list
```

LLVM Storage Model: Locals and alloca

□ Several kinds of storage:

- Local variables (or temporaries): **%uid**
- Global declarations (e.g. for string constants): **@gid**
- Abstract locations: stack-allocated storage created by the **alloca** instruction
- Heap-allocated structures created by external calls (e.g. to **malloc**)

□ Local variables: **%uid = ...**

- Intended to be an abstract version of machine registers.
- Must satisfy the SSA invariant in LLVM
 - Each %uid appears only once on the left-hand side of an assignment in the entire CFG.
 - The value of a %uid remains unchanged throughout its lifetime
 - Analogous to “let %uid = e in ...” in OCaml
- Not required in LLVM Lite

□ The **alloca** instruction: **%ptr = alloca typ**

- Gives an abstract version of stack slots
- The returned reference is stored in local
- The type determines the amount of space allocated
- Access the contents via the load and store instructions:

%acc = alloca i64 ; allocate a storage slot

store i64 341, i64* %acc ; store the integer value 341

%x = load i64, i64* %acc ; load the value 341 into %x

Datatypes in the LLVM Lite IR

□ LLVM's IR uses types to describe the structure of data

```
t ::=                                     Types
    void
    i1 | i8 | i64                       N-bit integers
    [<#elts> x t]                       arrays
    fty                                 function types
    {t1, t2, ... , tn}               structures
    t*                                  pointers
    %Tident                             named (identified) type

fty ::=                                 Function Types
    t (t1, .., tn)                  return, argument types
```

```
type ty =
| Void
| I1
| I8
| I64
| Ptr
| Array of int * ty
| Struct of ty list
| Fn of fn_ty
and fn_ty = {
    params : ty list;
    ret : ty
}
```

● <#elts> is an integer constant ≥ 0

● Structure types (may be recursive) can be named at the top level:

```
%T1 = type {t1, t2, ... , tn}
```

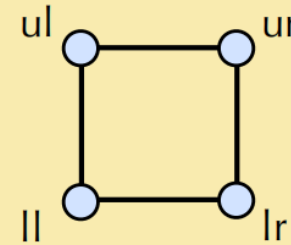
Compiling Structured Data

❑ Consider C-style structures like those below

```
struct Point { int x; int y; };

struct Rect  { struct Point ll, lr, ul, ur };

struct Rect mk_square(struct Point ll, int len) {
    struct Rect square;
    square.ll = square.lr = square.ul = square.ur = ll;
    square.lr.x += len;
    square.ur.x += len;
    square.ur.y += len;
    square.ul.y += len;
    return square;
}
```



❑ How do we represent **Point** and **Rect** values?

Representing and Accessing Structs

❑ Store the data using contiguous words of memory

- Represent a **Point** value **p** as the address of the first word

```
struct Point { int x; int y; };    p → 

|   |   |
|---|---|
| x | y |
|---|---|


```

- Store the data of **Rect** using 8 contiguous words of memory

```
struct Rect { struct Point ll, lr, ul, ur };  
square → 

|      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|
| ll.x | ll.y | lr.x | lr.y | ul.x | ul.y | ur.x | ur.y |
|------|------|------|------|------|------|------|------|


```

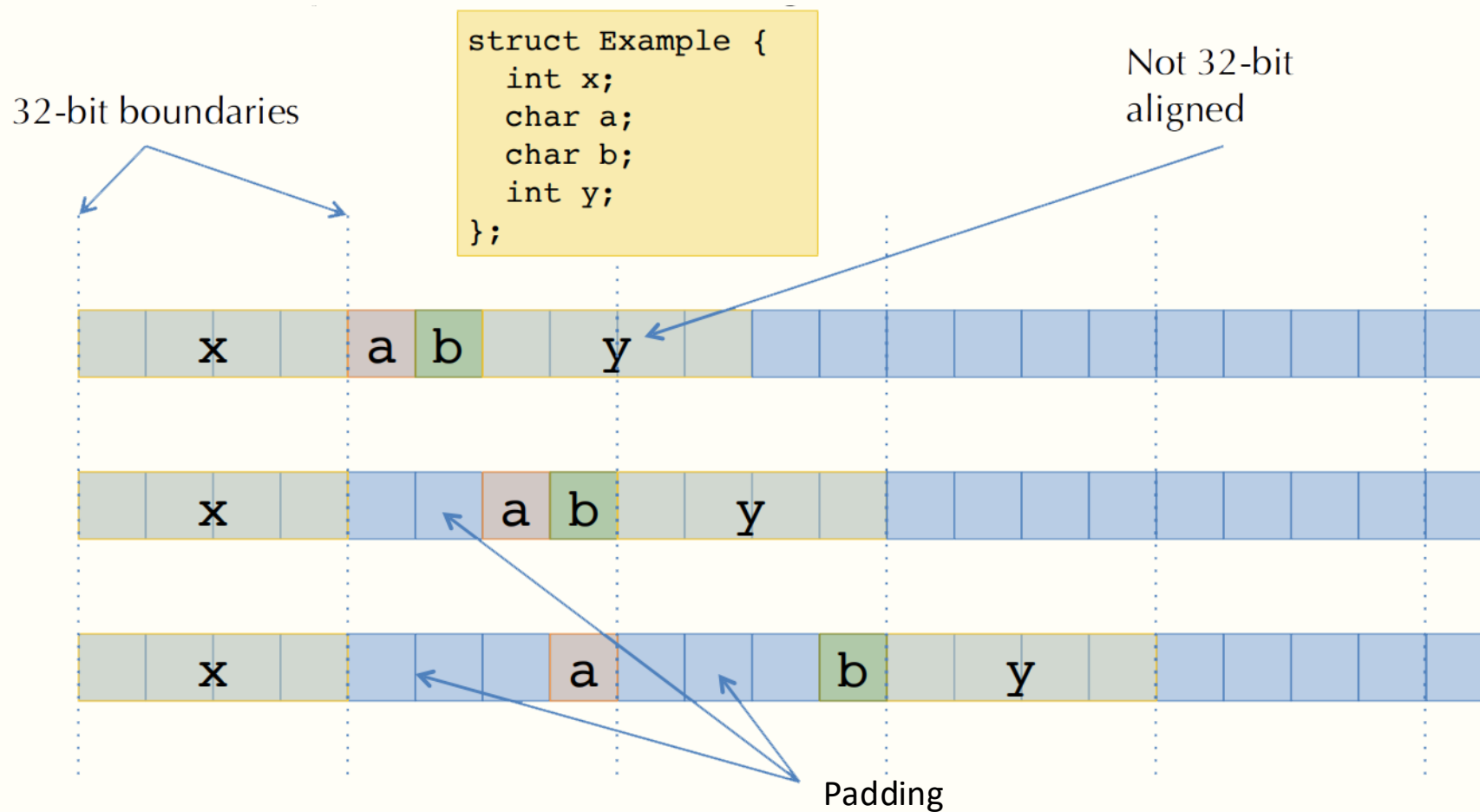
- Compiler needs to know the size of the struct to allocate the needed storage space.
- Compiler needs to know the shape of the struct to index into the structure.

❑ Consider Member Access `[[square.ul.y]]`:

- Assume that `%rcx` holds the base address of square
- Calculate the offset relative to the base pointer of the data:
 - `ul = sizeof(struct Point) + sizeof(struct Point) ; y = sizeof(int)`
- So: `[[square.ul.y]] = (Movq 20(%rcx) ans, ans)`

Padding & Alignment

❑ How to lay out non-homogeneous structured data?



Copy-in/Copy-out

❑ When we do an assignment in C, as in:

```
struct Rect mk_square(struct Point ll, int elen) {  
    struct Square res;  
    res.lr = ll;  
    ...  
}
```

● We copy all elements from the source and put them in the target.

❑ Same as doing word-level operations:

```
struct Rect mk_square(struct Point ll, int elen) {  
    struct Square res;  
    res.lr.x = ll.x;  
    res.lr.y = ll.x;  
    ...  
}
```

❑ For really large copies, the compiler uses something like **memcpy**

● which is implemented using a *loop* in **assembly**

C Procedure Calls

- ❑ When calling a procedure, we copy arguments in, and copy results out
 - Caller sets aside extra space in its frame to store results that don't fit in *%rax*
 - We do the same with scalar values such as integers or doubles.
 - Sometimes, this is termed “**call-by-value**”.
 - This is bad terminology
 - **Copy-in/copy-out** is more accurate
 - Benefit: locality
 - Problem: expensive for large records...
- ❑ In C: can opt to pass pointers to structs: “**call-by-reference**”
 - Languages like Java and OCaml always pass non-word-sized objects by reference
 - But note that returning references to stack-allocated data can cause problems...

Call-by-Reference

- ❑ Caller passes in the address of the point and the result (1 word each)

```
void mkSquare(struct Point *ll, int elen,
              struct Rect *res) {
    res->lr = res->ul = res->ur = res->ll = *ll;
    res->lr.x += elen;
    res->ur.x += elen;
    res->ur.y += elen;
    res->ul.y += elen;
}

void foo() {
    struct Point origin = {0,0};
    struct Square unit_sq;
    mkSquare(&origin, 1, &unit_sq);
}
```

```
int* bad() {
    int x = 341;
    int *ptr = &x;
    return ptr;
}
```



Returns a dangling pointer

- ❑ Stack pointers can escape

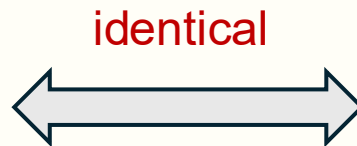
- For data that persists across a function call, allocate storage in the heap...
- In C, use the **malloc** library

Arrays

❑ Space is allocated on the stack for **buf**

- Note: without the ability to allocate stack space dynamically, the **alloca** function needs to know the size of **buf** at compile time...

```
void foo() {  
    char buf[27];  
  
    buf[0] = 'a';  
    buf[1] = 'b';  
    ...  
    buf[25] = 'z';  
    buf[26] = 0;  
}
```



```
void foo() {  
    char buf[27];  
  
    *(buf) = 'a';  
    *(buf+1) = 'b';  
    ...  
    *(buf+25) = 'z';  
    *(buf+26) = 0;  
}
```

❑ **buf[i]** is really just:

- $(\text{base_of_array}) + i * \text{element_size}$

Multi-dimensional Arrays

m[0][0]	m[0][1]	m[0][2]
m[1][0]	m[1][1]	m[1][2]
m[2][0]	m[2][1]	m[2][2]
m[3][0]	m[3][1]	m[3][2]

❑ In C, arrays are laid out in row-major order

● **int m[4][3]** yields an array with 4 rows and 3 columns

m[0][0]	m[0][1]	m[0][2]	m[1][0]	m[1][1]	m[1][2]	m[2][0]	m[2][1]	...
---------	---------	---------	---------	---------	---------	---------	---------	-----

● So m[i][j] is located where?

● (base address of m) + (i * 3 * sizeof(int)) + j * sizeof(int)

❑ In Fortran, arrays are laid out in column-major order

m[0][0]	m[1][0]	m[2][0]	m[3][0]	m[0][1]	m[1][1]	m[2][1]	m[3][1]	...
---------	---------	---------	---------	---------	---------	---------	---------	-----

❑ In ML, there are no multi-dimensional arrays

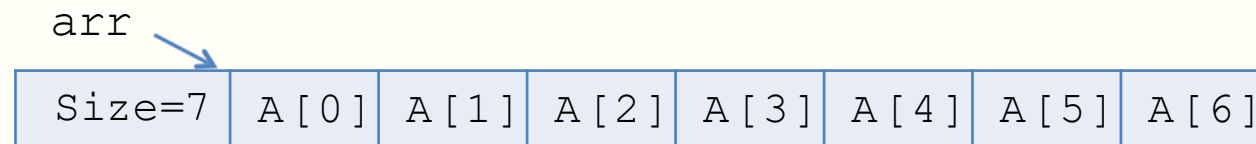
● **(int array) array** is represented as an array of pointers to arrays of ints

❑ Why is knowing the memory layout strategy important?

Array Bounds Checks

❑ Safe languages check array indices to ensure that they are in bounds

- e.g. Java, C#, ML but not C, C++
- Compiler generates code to test that the computed offset is legal
 - Needs to know the size of the array ... where to store it?
 - One answer: Store the size before the array contents.



- Other possibilities: Store size and a pointer to array data or only permits statically known array sizes (as in Pascal, very unwieldy in practice)
- Implementation: assume a0 holds the base (arr) and a1 holds the index i

```
movq -8(%rax) %rdx      // load size into rdx
cmpq %rdx %rcx          // compare index to bound
j  l __ok               // jump if 0 <= i < size
callq __err_oob         // test failed, call the error handler
__ok:
movq (%rax, %rcx, 8) dest // do the load from the array access
```

C-style Strings

❑ A string constant "foo" is represented as global data:

```
__string42: 0x66 0x6F 0x6F 0x00
```

- Strings are usually placed in the text segment, so they are read-only
- allows all copies of the same string to be shared

❑ Rookie mistake (in C): write to a string constant

```
char *p = "foo";  
p[0] = 'b';
```

❑ Instead, must allocate space on the heap:

```
char *p = (char *)malloc(4 * sizeof(char));  
strncpy(p, "foo", 4); /* include the null byte */  
p[0] = 'b';
```

Tagged Datatypes

❑ C-style Enumerations:

```
enum Day {sun, mon, tue, wed, thu, fri, sat} today;
```

❑ ML-style datatypes:

```
type day = Sun | Mon | Tue | Wed | Thu | Fri | Sat
```

❑ Associate an integer tag with each case: sun = 0, mon = 1, ...

● C lets programmers choose the tags

❑ ML datatypes can also carry data:

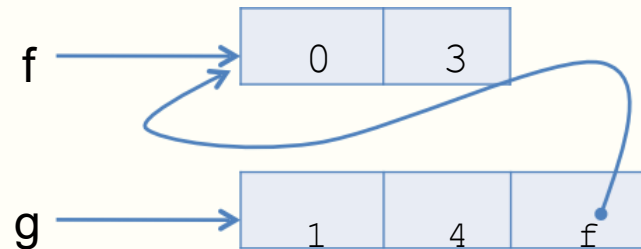
```
type foo = Bar of int | Baz of int * foo
```

● Representation: a foo value is a pointer to a pair: (tag, data)

❑ Example: tag(Bar) = 0, tag(Baz) = 1

$\llbracket \text{let } g = \text{Baz}(4, f) \rrbracket$

$= \llbracket \text{let } f = \text{Bar}(3) \rrbracket =$



Switch Compilation

□ $[[\text{switch}(e) \{ \text{case } \text{tag1}: s1; \text{break}; \text{case } \text{tag2}: s2; \dots \}]] =$

- Each $\$tag1 \dots \$tagN$ is just a constant int value

- $[[\text{break};]]$ (within the switch branches) is:

 - $\text{br } \%merge$

- What happens if

 - some of the break statements are omitted?

 - Control falls through to the next branch

 - i.e., omit the $\text{br } \%merge$ statement

```
switch (e) {  
    case sun: s1; break;  
    case mon: s2; break;  
    ...  
    case sat: s3; break;  
}
```

```
%tag = [[e]];  
br label %l1  
l1: %cmp1 = icmp eq %tag, $tag1  
    br %cmp1 label %b1, label %l2  
b1: [[s1]]  
    br label %merge  
l2: %cmp2 = icmp eq %tag, $tag2  
    br %cmp2 label %b2, label %l3  
b2: [[s2]]  
    br label %l3  
...  
lN: %cmpN = icmp eq %tag, $tagN  
    br %cmpN label %bN, label %merge  
bN: [[sN]]  
    br label %merge  
merge:
```

Alternatives for Switch Compilation

- ❑ Nested if-then-else works OK in practice if # of branches is small
 - e.g. < 16 or so
- ❑ For more branches, use better data structures to organize the jumps:
 - Create a table of pairs (v1, branch_label) and loop through
 - Or, do a binary search rather than a linear search
 - Or, use a hash table rather than a binary search
- ❑ One common case: the tags are dense in some range [min ... max]
 - Let $N = \text{max} - \text{min}$
 - Create a branch table `Branches[N]` where `Branches[i] = branch_label` for tag `i`.
 - Compute `tag = ⌊e⌋` and then do an **indirect jump: J Branches[tag]**
- ❑ Common to use heuristics to combine these techniques

ML-style Pattern Matching

❑ ML-style match statements are like C's switch statements except:

- Patterns can bind variables
- Patterns can nest

```
match e with
| Bar(z) -> e1
| Baz(y, Bar(w)) -> e2
| _ -> e3
```



```
match e with
| Bar(z) -> e1
| Baz(y, tmp) ->
    (match tmp with
     | Bar(w) -> e2
     | Baz(_, _) -> e3)
```

❑ Compilation strategy:

- “Flatten” nested patterns into matches against one constructor at a time
- Compile the match against the tags of the datatype, as for C-style switches
- Each branch additionally must copy data from $\llbracket e \rrbracket$ to the variables bound in the patterns

❑ There are many opportunities for optimization

- many papers about “pattern match compilation”
- Many of these transformations can be done at the AST level

Example LLVM Types

- ❑ An array of 341 integers: `[ 341 x i64 ]`
- ❑ A two-dimensional array of integers: `[ 3 x [ 4 x i64 ] ]`
- ❑ Structure for representing arrays with their length: `{i64,[0 x i64]}`
 - There is no array-bounds check;
 - The static type information is only used for calculating pointer offsets.
- ❑ C-style linked lists (declared at the top level):

```
%Node = type { i64, %Node* }
```

- ❑ Structs from the C program shown earlier:

```
%Rect = { %Point, %Point, %Point, %Point }  
%Point = { i64, i64 }
```

LLVM `getelementptr` Instruction

❑ “type `insn = ... | Gep of ty * opn * typed_opn list`” in LLVM Lite

❑ Compute pointer address:

- Given a pointer and a “path” through the structured data pointed to by that pointer, *getelementptr* computes an address
- This is the abstract analogue of the X86 LEA (load effective address)
 - It does not access memory
 - It is a “type-indexed” operation, since the size computations depend on the type

```
insn ::= ... | getelementptr t* %val, t1 idx1, t2 idx2 ,...
```

❑ Example: access the x component of the first point of a rectangle:

```
%tmp1 = getelementptr %Rect* %square, i32 0, i32 0
```

```
%tmp2 = getelementptr %Point* %tmp1, i32 0, i32 0
```

```
struct Point { int x; int y; };
```

❑ GEP never dereferences the address it’s calculating:

- GEP only produces pointers by doing arithmetic
- It doesn’t actually traverse the links of a data structure

```
struct Rect { struct Point ll, lr, ul, ur };
```

❑ To index into a deeply nested structure, need to “follow the pointer” by loading from the computed pointer

GEP Example

```
struct RT {
    int A;
    int B[10][20];
    int C;
}
struct ST {
    struct RT X;
    int Y;
    struct RT Z;
}
int *foo(struct ST *s) {
    return &s[1].Z.B[5][13];
}
```

1. %s is a pointer to an (array of) %ST structs, suppose the pointer value is ADDR

2. Compute the index of the 1st element by adding size_ty(%ST).

3. Compute the index of the z field by adding size_ty(%RT) + size_ty(i32) to skip past X and Y.

4. Compute the index of the B field by adding size_ty(i32) to skip past A.

5. Index into the 2d array.

```
%RT = type { i32, [10 x [20 x i32]], i32 }
%ST = type { %RT, i32, %RT }
define i32* @foo(%ST* %s) {
entry:
    %arrayidx = getelementptr %ST* %s, i32 1, i32 2, i32 1, i32 5, i32 13
    ret i32* %arrayidx
}
```

Final answer: ADDR + size_ty(%ST) + size_ty(%RT) + size_ty(i32)
+ size_ty(i32) + 5*20*size_ty(i32) + 13*size_ty(i32)

Compiling Data Structures via LLVM

❑ Translate high-level language types into an LLVM representation type

- For some languages (e.g. C) this process is straightforward
 - The translation simply uses platform-specific alignment and padding
- For other languages (e.g. OO languages), the elaboration might be complex
 - e.g. for OCaml, array types might be translated to pointers to length-indexed structs

`[[int array]] = { i32, [0 x i32]}*`

❑ Translate accesses of the data into `getelementptr` operations:

- e.g. for Ocaml array size access:

`[[length a]] := %1 = getelementptr {i32, [0xi32]}* %a, i32 0, i32 0`

❑ **Bitcast:** misuse can cause serious bugs

```
%rect2 = type { i64, i64 }           ; two-field record
%rect3 = type { i64, i64, i64 }      ; three-field record

define @foo() {
  %1 = alloca %rect3                ; allocate a three-field record
  %2 = bitcast %rect3* %1 to %rect2* ; safe cast
  %3 = getelementptr %rect2* %2, i32 0, i32 1 ; allowed
  ...
}
```

作业：熟悉LLVMLite IR

- LLVMLite IR: see IR.ml
- IRPrinter.ml: 可以打印出类似LLVM的.ll格式的IR
- Main.ml: 包含有一些函数的IR实例, 其打印形式放在examples.ir
- 请同学们学习以上代码, 仿照Main.ml, 用我们的IR编写一些函数, 要求函数体覆盖我们的IR中的绝大多数指令, 并调用IRPrinter.ml中相关函数将IR的文本形式打印出来。

Project: Compiling LLVMlite to x86lite

□ LLVMlite is a small subset of the LLVM IR

● <https://groups.seas.harvard.edu/courses/cs153/2019fa/llvmlite.html>